Universitatea „Petrol – Gaze” din Ploiești

Optimizare liniară

Realizat de: Coșereanu Emanuel, INFO an 3, grupa 40322

Cuprins

Problema 1 1
Modelarea problemei 1
Variabile 1
Restricții 1
Funcție obiectiv 2
Caracterizarea problemei 2
Optimizare matematică 2
Optimizare statică 2
Optimizare parametrică 2
Optimizare cu restricții 2
Programare discretă 2
Programare în numere întregi 2
Programare liniară 3
Programare convexă 3
Programare deterministă 3
Optimizare uniobiectiv 3
Dimensiunea spațiului de stocare 3
Analiză folosind al doilea set de restricții 3
Analiză folosind primul set de restricții 4
Modelare cu PuLP 5
Date de ieșire obținute 6
Analiza datelor obținute 6
Schimbarea instanțelor 6
Tabel pentru datele de intrare 6
Tabel pentru restricții 7
Tabel pentru date de ieșire 7
Observații asupra datelor obținute 8
Implementări în plus 9

Implementare Branch and Bound 9
Implementare Hill Climbing 12
Implementare Simulated Annealing 13
Concluzii finale 15
Centralizarea datelor (costul minim găsit) 15
Analiza pe metode 15

Problema 1

Un nutriționist dorește să obțină un plan alimentar zilnic, bazat pe patru alimente, care toate trebuie să apară în plan.

- Alimentele: chiflă, pește, iaurt, măr;
- Planul să conțină maxim 250 g de chiflă, 300 g de măr și 300 g de iaurt;
- Costul alimentelor per 100 de grame: chiflă – 3 lei, pește – 8 lei, iaurt – 4 lei, măr – 2 lei;
- Cantitatea nutrițională zilnică minim necesară: 2.000 kcal, 70g proteine, 40g carbohidrați;
- Valorile nutriționale asociate:

Alimentul (100g)	Calorii (kcal)	Proteine (g)	Carbohidrați (g)
Chiflă	255	5	48,3
Pește	172	25	0
Iaurt	100	9	4,7
Măr	152	0,8	33,8

Cerința: Să se determine cantitățile din fiecare aliment care să fie incluse în plan astfel încât costul total al planului să fie minim.

Modelarea problemei

Variabile

Xc – cantitatea in grame a chiflelor

Xp – cantitatea in grame a peștelui

Xi – cantitatea in grame a iaurtului

Xm – cantitatea in grame a mărului

Restricții

Setul 1 de restricții

(R1) 255 / 100 * Xc + 172 / 100 * Xp + 100 / 100 * Xi + 152 / 100 * Xm >= 2000 //calorii

(R2) 5 / 100 * Xc + 25 / 100 * Xp + 9 / 100 * Xi + 0.8 / 100 * Xm >= 70 //proteine

(R3) 48.3 / 100 * Xc + 0 / 100 * Xp + 4.7 / 100 * Xi + 33.8 / 100 * Xm >= 40 //carbohidrati

Setul 2 de restricții

(R4) Xc >= 1

(R5) Xc <= 250

(R6) Xm >= 1

(R7) Xm <= 300

(R8) Xi >= 1

(R9) Xi <= 300

(R10) Xp >= 1

Funcție obiectiv

Funcția obiectiv a acestei probleme urmărește minimizarea costului total al dietei.

Min(3 * Xc/100 + 8 * Xp/100 + 4 * Xi/100 + 2 * Xm / 100**) – costul planului**

Caracterizarea problemei

Optimizare matematică

Este optimizare matematică, deoarece se cunoaște explicit funcția obiectiv și restricțiile, nu se bazează pe simulări sau teste empirice.

Optimizare statică

Este optimizare statică, întrucât datele de intrare nu se modifică de la începerea rezolvării problemei până la găsirea soluției.

Optimizare parametrică

Este optimizare parametrică, deoarece variabilele (Xc, Xp, Xi, Xm) sunt scalari, deci funcția obiectiv are parametri scalari, nu parametri funcții.

Optimizare cu restricții

Este optimizare cu restricții, existând 10 restricții - unele explicite, altele implicite prin natura problemei.

Programare discretă

Este programare discretă, întrucât toate variabilele de decizie (necunoscuta compusă) trebuie să primească valori discrete (dintr-o mulțime finită sau numărabilă) - număr de grame, care sunt numere întregi.

Programare în numere întregi

Este programare în numere întregi (caz particular al programării discrete): Xc, Xp, Xi, Xm $\in \mathbb{Z} \geq 0$ (nu există variabile continue, toate sunt numere întregi).

Programare liniară

Este programare liniară, pentru că funcția obiectiv este liniară, restricțiile sunt și ele liniare (sunt combinații liniare ale parametrilor).

Programare convexă

Este programare convexă pentru că funcția obiectiv este convexă (fiind liniară - orice funcție liniară este convexă), iar mulțimea fezabilă este definită de inegalități liniare.

Programare deterministă

Este programare deterministă întrucât toate datele de intrare (costuri, calorii, proteine, carbohidrați) sunt cunoscute apriori și nu intervin variabile aleatoare.

Optimizare uniobiectiv

Este optimizare uniobiectiv, deoarece există un singur obiectiv urmărit: minimizarea prețului. Nu se cere optimizarea simultană a mai multor obiective.

Dimensiunea spațiului de stocare

Cerința: Care este dimensiunea spațiului de căutare (numărul sau ordinul de mărime al numărului de soluții valide) pentru instanța din enunț (și pentru problema generală)? Se pot identifica toate soluțiile fezabile în timp rezonabil?

Analiză folosind al doilea set de restricții

Pentru prima analiză s-a folosit setul 2 de restricții

Xc e o valoare din mulțimea {1, 2, ..., 250} – total 250 valori

Xm e o valoare din mulțimea {1, 2, ..., 300} – total 300 valori

Xi e o valoare din mulțimea {1, 2, ..., 300} – total 300 valori

Xp e o valoare din mulțimea {1, 2, ..., 1162} – total 1162 valori (pentru peste s-a ținut cont de setul 1)

100g peste .... 25g proteina

Xg peste .... 70g proteina

Xg = 70 * 100 / 25 = 280 grame de peste

100g peste .... 172g kcal

Xg peste .... 2000 kcal

Xg = 200000 / 172 = 1162 grame de peste

Ținând cont de setul 2 de restricții s-a obținut următoarea dimensiune a spațiului de căutare:

Totalul soluțiilor fezabile = 250 * 300 * 300 * 1162 = 26 145 000 000, adică de ordinul O(10 ^ 10)

Analiză folosind primul set de restricții

Pentru a doua analiză s-a folosit setul 1 de restricții

255 / 100 * Xc + 172 / 100 * Xp + 100 / 100 * Xi + 152 / 100 * Xm >= 2000 //calorii

Xc_min = 2000 / 2,55 = 784 grame de chifle

Xp_min = 2000 / 1,72 = 1162 grame de peste

Xi_min = 2000 / 1 = 2000 grame de iaurt

Xm_min = 2000 / 1,52 = 1315 grame de măr

5 / 100 * Xc + 25 / 100 * Xp + 9 / 100 * Xi + 0.8 / 100 * Xm >= 70 //proteine

Xc_min = 70 / 0,05 = 1400 grame de chifle

Xp_min = 70 / 0,25 = 280 grame de peste

Xi_min = 70 / 0,09 = 777 grame de iaurt

Xm_min = 70 / 0,008 = 8750 grame de măr

48.3 / 100 * Xc + 0 / 100 * Xp + 4.7 / 100 * Xi + 33.8 / 100 * Xm >= 40 //carbohidrati

Xc_min = 40 / 0,483 = 82 grame de chifle

Xp_min = 40 / 0 = infinit grame de peste

Xi_min = 40 / 0.047 = 851 grame de iaurt

Xm_min = 40 / 0,388 = 118 grame de măr

Xc aparține {1, ..., Max(250, 784, 1400, 82) = 1400} – total 1400 de valori

Xp aparține {1, ..., Max(1162, 280, infinit) = 1162} – total 1162 de valori (nu tinem cont de infinit)

Xi aparține {1, ..., Max(300, 2000, 777, 851) = 2000} – total 2000 de valori

Xm aparține {1, ..., Max(300, 1315, 8750, 118) = 8750} – total 8750 de valori

---

Ținând cont de setul 1 de restricții s-a obținut următoarea dimensiune a spațiului de căutare:

Totalul soluțiilor fezabile = 1400 * 1162 * 2000 * 8750 = 28 469 000 000 000 adică de ordinul O(10 ^ 14).

Modelare cu PuLP

Cerința: **Modelați problema cu PuLP (adăugând comentarii) și apelați un solver pentru a o rezolva. Interpretați datele de ieșire și analizați-le.**

```
import pulp
model_1 = pulp.LpProblem("Problema 1", pulp.LpMinimize)

Xc = pulp.LpVariable("Xc", lowBound=1, upBound=250,
cat="Integer")
Xp = pulp.LpVariable("Xp", lowBound=1, cat="Integer")
Xi = pulp.LpVariable("Xi", lowBound=1, upBound=300,
cat="Integer")
Xm = pulp.LpVariable("Xm", lowBound=1, upBound=300,
cat="Integer")

model_1 += 3 * Xc/100 + 8 * Xp/100 + 4 * Xi/100 + 2 * Xm / 100,
"Min(3 * Xc/100 + 8 * Xp/100 + 4 * Xi/100 + 2 * Xm / 100)"

model_1 += 255 / 100 * Xc + 172 / 100 * Xp + 100 / 100 * Xi +
152 / 100 * Xm >= 2000, "restrictie 1 calorii"

model_1 += 5 / 100 * Xc + 25 / 100 * Xp + 9 / 100 * Xi + 0.8 /
100 * Xm >= 70, "restrictie 2 proteine"

model_1 += 48.3 / 100 * Xc + 0 / 100 * Xp + 4.7 / 100 * Xi +
33.8 / 100 * Xm >= 40, "restrictie 3 carbohidrati"

print(model_1)
status = model_1.solve()
print(f"Status solver: {pulp.LpStatus[status]}")
print(f"Solutiile: Xc={pulp.value(Xc)}, Xp={pulp.value(Xp)},
Xi={pulp.value(Xi)}, Xm={pulp.value(Xm)}, Min(3 * Xc/100 + 8 *
```

```
Xp/100 + 4 * Xi/100 + 2 * Xm /
100)={pulp.value(model_1.objective)}")
```

Date de ieșire obținute

Datele de ieșire obținute în urma modelării problemei și apelării unui solver sunt:

Produs	Chifle	Peste	Iaurt	Mere
Cantitate (g)	250	353	300	300
Preț total	53.74 RON			

Analiza datelor obținute

Din aceste date putem observa că pentru:

- **250g chifle** avem 637,5 calorii, 12,5g proteină, 120,75g carbohidrați.
- **353g peste** avem 607,16 calorii, 88,25g proteină, 0g carbohidrați.
- **300g iaurt** avem 300 calorii, 27g proteină, 14,1g carbohidrați.
- **300g mere** avem 456 calorii, 2,4g proteină, 101,4g carbohidrați.

Pentru cantitățile alese de solver avem următoarele valori pentru numărul total de calorii din dietă, respectiv, pentru numărul de grame de proteină și carbohidrați.

	Calorii (kcal)	Proteine (g)	Carbohidrați (g)
Obiectiv (valori minime necesare)	2000	70	40
Soluție optima (valori obținute)	2000,66	130,15	236,25

Schimbarea instanțelor

Cerința: **Schimbați instanța (valorile datelor de intrare, inclusiv pentru restricții) pentru a vedea cum se modifică datele de ieșire, inclusiv status-ul rezolvării. Înscrieți într-un tabel aceste valori și noile rezultate. Ce observați?**

Tabel pentru datele de intrare

Datele modificate vor fi colorate, daca în seturile de date următoare se găsește o culoare din setul anterior, înseamnă că se folosește atributul respectiv cu aceeași valoare ca la setul anterior, dacă celulele nu sunt colorate înseamnă că s-au folosit aceleași valori din setul de date din problemă.

Setul de date	Alimentul (100g)	Calorii (kcal)	Proteine (g)	Carbohidrați (g)
1	Chiflă	255	5	48,3
	Pește	172	25	1,2

Setul de date	Alimentul (100g)	Calorii (kcal)	Proteine (g)	Carbohidrați (g)
	Iaurt	100	9	4,7
	Măr	152	0,8	33,8
2	Chiflă	255	5	48,3
	Pește	172	25	1,2
	Iaurt	100	9	4,7
	Măr	152	0,8	33,8
3	Chiflă	255	5	48,3
	Pește	110	25	1,2
	Iaurt	100	9	4,7
	Măr	152	0,8	33,8
4	Chiflă	255	5	48,3
	Pește	110	25	1,2
	Iaurt	60	9	4,7
	Măr	152	0,8	33,8
5	Chiflă	255	5	48,3
	Pește	110	25	1,2
	Iaurt	60	9	4,7
	Măr	52	0,8	33,8

Tabel pentru restricții

S-au schimbat doar valorile, semnele au rămas la fel.

Se aplică aceeași regulă de colorare ca la tabelul de mai sus.

Setul de date	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
1	2000	70	40	1	250	1	300	1	300	1
2	2100	80	40	1	250	1	300	1	300	1
3	2100	80	50	1	250	1	300	1	300	1
4	2100	80	50	1	250	1	300	1	400	1
5	2500	80	50	1	250	1	300	1	400	1

Tabel pentru date de ieșire

Setul de date	Produs	Chifle	Peste	Iaurt	Mere
1	**Cantitate (g)**	250	353	300	300
	Preț total	53.74 RON			
2	**Cantitate (g)**	250	411	300	300
	Preț total	58.38 RON			
3	**Cantitate (g)**	250	643	300	300

Setul de date	Produs	Chifle	Peste	Iaurt	Mere
	Preț total	76.94 RON			
4	**Cantitate (g)**	250	697	400	300
	Preț total	85.26 RON			
5	**Cantitate (g)**	250	1334	399	300
	Preț total	136.18 RON			

Observații asupra datelor obținute

Tabel de analiză a datelor obținute calculând numărul total de calorii, gramele de proteine și carbohidrați al dietelor.

În construirea observațiilor se va ține cont și de prețul dietei, preț afișat în tabelele de mai sus.

Setul de date		Calorii (kcal)	Proteine (g)	Carbohidrați (g)
1	**Obiectiv (valori minime necesare)**	2000	70	40
	Soluție optimă (valori obținute)	2000,66	130,15	240,486
2	**Obiectiv (valori minime necesare)**	2100	80	40
	Soluție optimă (valori obținute)	2100,42	135,65	241,182
3	**Obiectiv (valori minime necesare)**	2100	80	50
	Soluție optimă (valori obținute)	2100,8	202,65	243,96
4	**Obiectiv (valori minime necesare)**	2100	80	50
	Soluție optimă (valori obținute)	2100,2	225,15	292,814
5	**Obiectiv (valori minime necesare)**	2500	80	50
	Soluție optimă (valori obținute)	2500,3	384,31	256,911

În urma valorilor obținute de către solver și în urma valorilor calculate de mine pentru numărul total de calorii din diete, respectiv numărul gramelor de proteine și de carbohidrați se poate concluziona că cea mai eficientă dietă atât din punct de vedere al numărului de calorii, numărului de grame de proteine și numărului de grame de carbohidrați este dieta numărul 2 care pentru un preț de doar 58,38 RON ne oferă 2100 de kcal (deoarece dorim să slăbim într-un mod cât mai ieftin și sănătos) cu 135,65 grame de proteină și 241,182 grame de carbohidrați ce ne acoperă necesarul zilnic de nutrienți.

În comparație cu prima dietă ne aduce un plus de proteină necesar păstrării musculaturii și un plus de calorii care deși ne încetinește puțin procesul de slăbire ne ajută să fim mai energici.

În comparație cu dietele 3 și 4 avem un preț mai mic dar cu aproape aceleași beneficii, singurul avantaj ar celor două este numărul mare de grame de proteină, dar dezavantajul este numărul mare de carbohidrați.

În comparație cu dieta 5, se poate observa clar prețul izbitor de mare pentru numărul de calorii pe care îl avem, cantitatea de pește ce trebuie consumată într-o zi și numărul de grame de proteine care, deși este foarte mare, nu ne aduce un beneficiu.

Implementări în plus

Cerințe: **Implementați în plus pentru rezolvarea problemei:**

a. **un algoritm clasic de optimizare dintre (dacă este adecvat problemei): - greedy, branch-and-bound, programare dinamică**
b. **un algoritm dintre (dacă este adecvat problemei): - metoda gradientului, metoda Newton (cu repornire), hill climbing**
c. **un algoritm dintre (dacă este adecvat problemei): - simulated annealing, tabu search**

Cei bazați pe valori aleatoare se vor executa de câte 3 ori pentru aceleași valori ale parametrilor (pe care le puteți varia în 3-5 moduri) și se vor înregistra toate rezultatele.

Implementare Branch and Bound

Înainte ca problema să fie rezolvată cu Branch and Bound s-a făcut o analiză a celorlalte două tehnici de programare (greedy și programare dinamică) și s-a realizat că nicio altă metodă nu ne rezolvă problema corect din următoarele motive:

- greedy ar presupune să alegem mereu alimentul cu cel mai bun raport cost/kcal, sau cost/proteine, ori cost/carbohidrați. Pentru că avem 3 nutrienți diferiți, iar un aliment poate fi foarte bun la unul (ex: peștele are foarte multă proteină) și slab la altul (ex:

peștele are puțini carbohidrați) nu există un criteriu unic care să ordoneze elementele corect. De aici rezultă că metoda greedy nu ar genera o rezolvare optimă.

- programarea dinamică ar trebui să țină cont numărul de calorii, cantitatea de proteine și carbohidrați rezultând într-o programare dinamică tridimensională dp[2000][70][40], iar algoritmul va deveni prea lent și va consuma prea multă memorie. Programarea dinamică este teoretic posibil de implementat, dar practic imposibil.

Astfel, tehnica branch and bound ne rezolvă problema corect, deoarece problema este de tip ILP (integer linear programming), cu constrângeri liniare, cu obiectiv liniar, adică exact tipul de problemă pe care branch and bound se pliază cel mai bine. Mai jos este prezentat algoritmul acestei tehnici pentru problema noastră.

Soluția obținută în urma rulării algoritmului de mai jos este aceeași ca soluția dată de către solver.

```
import pulp
import copy

#rezolva LP (Linear Programming) relaxat pentru un set de limite
def solve_lp(bounds):
    model = pulp.LpProblem("node", pulp.LpMinimize)

    # variabile continue (valorile nu sunt intregi)
    Xc = pulp.LpVariable("Xc", lowBound=bounds["Xc"][0],
upBound=bounds["Xc"][1])
    Xp = pulp.LpVariable("Xp", lowBound=bounds["Xp"][0],
upBound=bounds["Xp"][1])
    Xi = pulp.LpVariable("Xi", lowBound=bounds["Xi"][0],
upBound=bounds["Xi"][1])
    Xm = pulp.LpVariable("Xm", lowBound=bounds["Xm"][0],
upBound=bounds["Xm"][1])

    #functie obiectiv
    model += 3*Xc/100 + 8*Xp/100 + 4*Xi/100 + 2*Xm/100

    #constrangeri
    model += 255/100*Xc + 172/100*Xp + 100/100*Xi + 152/100*Xm >= 2000
    model += 5/100*Xc + 25/100*Xp + 9/100*Xi + 0.8/100*Xm >= 70
    model += 48.3/100*Xc + 0*Xp + 4.7/100*Xi + 33.8/100*Xm >= 40

    #rezolvare LP
    status = model.solve(pulp.PULP_CBC_CMD(msg=0))

    if status != 1:
        #LP infezabil sau fara solutie optima
        return None, None

    sol = {
        "Xc": pulp.value(Xc),
        "Xp": pulp.value(Xp),
```

```
        "Xi": pulp.value(Xi),
        "Xm": pulp.value(Xm)
    }

    return pulp.value(model.objective), sol


# verifica daca toate variabilele sunt intregi
def is_integer_solution(sol, eps=1e-6):
    for v in sol.values():
        if abs(v - round(v)) > eps:
            return False
    return True


#algoritmul branch and bound
def branch_and_bound():
    best_value = float("inf")
    best_solution = None

    #limite initiale pentru variabile
    root_bounds = {
        "Xc": (1, 250),
        "Xp": (1, None),
        "Xi": (1, 300),
        "Xm": (1, 300)
    }

    #folosim o stiva pentru explorare
    stack = [root_bounds]

    while stack:
        bounds = stack.pop()

        #rezolvam LP relaxat pentru nodul curent
        obj, sol = solve_lp(bounds)

        if sol is None:
            #nod infezabil
            continue

        #bounding: daca obiectivul e deja mai mare decat cel mai bun gasit,
ignoram nodul
        if obj >= best_value:
            continue

        #daca solutia LP este intreaga, actualizam best
        if is_integer_solution(sol):
            if obj < best_value:
                best_value = obj
                best_solution = {k: int(round(v)) for k, v in sol.items()}
            continue

        #alegem prima variabila fractionara pentru branching
        frac_var = None
        for var, val in sol.items():
            if abs(val - round(val)) > 1e-6:
```

```
                frac_var = var
                break

        #daca nu exista variabile fractionare, trecem mai departe
        if frac_var is None:
            continue

        val = sol[frac_var]
        floor_val = int(val)
        ceil_val = floor_val + 1

        #cream doua ramuri: var <= floor si var >= ceil
        low_bounds = copy.deepcopy(bounds)
        high_bounds = copy.deepcopy(bounds)

        #actualizam limitele pentru variabila aleasa
        low_bounds[frac_var] = (bounds[frac_var][0], floor_val)
        high_bounds[frac_var] = (ceil_val, bounds[frac_var][1])

        #adaugam ramurile in stiva
        stack.append(low_bounds)
        stack.append(high_bounds)

    return best_solution, best_value


#rulare
if __name__ == "__main__":
    sol, val = branch_and_bound()
    print("solutie branch and bound:")
    print(sol)
    print("valoare minima:", val)
```

Implementare Hill Climbing

Hill Climbing este metoda adecvată pentru rezolvarea problemei mele, deoarece poate lucra cu valori discrete (adică gramajele ingredientelor), nu necesită derivata și permite găsirea unei soluții bune (optim local) într-un spațiu mic de căutare (adică doar 4 variabile).

```
import random

#definim functia obiectiv
def objective_function(Xc, Xp, Xi, Xm):
    return 3 * Xc/100 + 8 * Xp/100 + 4 * Xi/100 + 2 * Xm / 100

#definim restrictiile (returneaza true daca sunt respectate)
def is_feasible(Xc, Xp, Xi, Xm):
    c1 = 2.55 * Xc + 1.72 * Xp + 1.00 * Xi + 1.52 * Xm >= 2000
    c2 = 0.05 * Xc + 0.25 * Xp + 0.09 * Xi + 0.008 * Xm >= 70
    c3 = 0.483 * Xc + 0 * Xp + 0.047 * Xi + 0.338 * Xm >= 40

    #restrictii de domeniu
    bounds = (1 <= Xc <= 250) and (1 <= Xp) and (1 <= Xi <= 300) and (1 <= Xm
<= 300)
```

```
    return c1 and c2 and c3 and bounds

#parametrii algoritmului
max_iterations = 10000
step_size = 5 #cu cat urcam sau coboram

#generam o solutie initiala, cautand-o pana obtinem una fezabila
current_sol = {"Xc": 150, "Xp": 150, "Xi": 150, "Xm": 150} #valori de start
while not is_feasible(**current_sol):
    current_sol = {
        "Xc": random.randint(1, 250),
        "Xp": random.randint(1, 500),
        "Xi": random.randint(1, 300),
        "Xm": random.randint(1, 300)
    }

current_fitness = objective_function(**current_sol)

#bucla principala hill climbing
for i in range(max_iterations):
    #cream o alterare (vecinatate)
    new_sol = current_sol.copy()
    var_to_change = random.choice(list(new_sol.keys()))
    new_sol[var_to_change] += random.randint(-step_size, step_size)

    #daca noua solutie e valida si mai buna (pretul este mai mic), o pastram
    if is_feasible(**new_sol):
        new_fitness = objective_function(**new_sol)
        if new_fitness < current_fitness:
            current_sol = new_sol
            current_fitness = new_fitness

print("--- Rezultat Hill Climbing ---")
print(f"Solutie: {current_sol}")
print(f"Cost minim gasit: {current_fitness}")
```

Deoarece se începe mereu de la o soluție candidat aleatoare, s-a rulat algoritmul de 3 ori obținându-se valorile din tabelul de mai jos.

Nr. rulare	Produs	Chifle	Peste	Iaurt	Mere
1	**Cantitate (g)**	248	485	125	269
	Preț total	56.62 RON			
2	**Cantitate (g)**	227	479	260	222
	Preț total	59.97 RON			
3	**Cantitate (g)**	233	440	268	251
	Preț total	57.93 RON			

Implementare Simulated Annealing

Metoda Simulated Annealing permite metodei alpinistului aplicată anterior să iasă din minimele locale. Aceasta pornește tot de la o soluție candidat oarecare căreia i se aplică o alterare ulterioară. Am ales Simulated Annealing în pofida căutării tabu (Tabu Search),

deoarece aceasta folosește o „temperatură” controlată care îi permite să accepte pentru moment soluții mai scumpe pentru a evita blocarea în optimele locale. Codul va fi prezentat mai jos.

```
import random
import math

#definim functia obiectiv
def objective_function(Xc, Xp, Xi, Xm):
    return 3 * Xc/100 + 8 * Xp/100 + 4 * Xi/100 + 2 * Xm / 100

#definim restrictiile
def is_feasible(Xc, Xp, Xi, Xm):
    c1 = 2.55 * Xc + 1.72 * Xp + 1.00 * Xi + 1.52 * Xm >= 2000
    c2 = 0.05 * Xc + 0.25 * Xp + 0.09 * Xi + 0.008 * Xm >= 70
    c3 = 0.483 * Xc + 0 * Xp + 0.047 * Xi + 0.338 * Xm >= 40
    bounds = (1 <= Xc <= 250) and (1 <= Xp) and (1 <= Xi <= 300) and (1 <= Xm
<= 300)
    return c1 and c2 and c3 and bounds

#parametrii simulated annealing
temp = 10.0          #temperatura initiala (mare => accepta multe erori)
cooling_rate = 0.999 #rata de racire (cat de repede scade T)
min_temp = 0.001     #pragul la care ne oprim
step_size = 10       #alterarea
iterations = 0       #contor

#solutia initiala
current_sol = {"Xc": 150, "Xp": 150, "Xi": 150, "Xm": 150}
while not is_feasible(**current_sol):
    current_sol = {
        "Xc": random.randint(1, 250), "Xp": random.randint(1, 500),
        "Xi": random.randint(1, 300), "Xm": random.randint(1, 300)
    }

current_fitness = objective_function(**current_sol)
best_sol = current_sol.copy()
best_fitness = current_fitness

#bluca principala Simulated Annealing
while temp > min_temp:
    iterations += 1

    #cream o alterare
    new_sol = current_sol.copy()
    var_to_change = random.choice(list(new_sol.keys()))
    new_sol[var_to_change] += random.randint(-step_size, step_size)

    if is_feasible(**new_sol):
        new_fitness = objective_function(**new_sol)

        #calculam diferenta de energie (cost)
        # Delta E > 0 inseamna ca solutia obtinuta este mai slaba (cost mai
mare)
        delta_e = new_fitness - current_fitness
```

```
        #regula de acceptare:
        #daca e mai buna (delta_e < 0), o acceptam mereu
        #daca e mai slaba, o acceptam cu o probabilitate e^(-delta_e / T)
        if delta_e < 0 or random.random() < math.exp(-delta_e / temp):
            current_sol = new_sol
            current_fitness = new_fitness

            #pastram separat cea mai buna solutie gasita vreodata
            if current_fitness < best_fitness:
                best_sol = current_sol.copy()
                best_fitness = current_fitness

    #racim sistemul
    temp *= cooling_rate

print(f"--- Rezultat Simulated Annealing (dupa {iterations} iteratii) ---")
print(f"Cea mai buna soluție: {best_sol}")
print(f"Cost minim absolut: {best_fitness}")
```

Deoarece se începe mereu de la o soluție candidat aleatoare, s-a rulat algoritmul de 3 ori obținându-se valorile din tabelul de mai jos.

Nr. rulare	Produs	Chifle	Peste	Iaurt	Mere	Nr. iterații
1	**Cantitate (g)**	250	375	262	300	9206
	Preț total	53.98 RON				
2	**Cantitate (g)**	250	355	296	300	9206
	Preț total	53.739 RON				
3	**Cantitate (g)**	250	361	286	300	9206
	Preț total	53.819 RON				

Concluzii finale

Mai jos va fi prezentată o analiză comparativă a rezultatelor obținute.

Centralizarea datelor (costul minim găsit)

Metoda de optimizare	Cel mai bun cost găsit	Statusul soluției
PuLP (solver exact)	53.74 RON	Optim global
Branch and Bound	53.74 RON	Optim global
Simulated Annealing	53.739 RON	Aproape de optim
Hill Climbing	56,62 RON	Sub-optim (optim local)

Analiza pe metode

Metodele exacte: PuLP și Branch and Bound:

a. **Performanță:** Acestea au obținut rezultatul ideal

b. **Soluția optimă:** Consumul maxim de chifle (250g), iaurt (300g) și mere (300g), completat cu cantitatea strict necesară de pește (353g) pentru a satisface necesarul proteic.
c. **Concluzie:** Branch and Bound este metoda teoretică cea mai eficientă iar PuLP o implementează eficient.

Simulated Annealing

a. **Performanță:** Metoda a dat un randament foarte bun reușind să găsească o soluție foarte apropiată de optimul global.
b. **Avantaj:** Datorită mecanismului de „temperatură”, algoritmul a putut „sări” peste minimele locale.
c. **Observație:** Rezultatele variază ușor între rulări => natura stocastică (aleatoare) a problemei.

Hill Climbing

a. **Performanță:** Cea mai slabă dintre cele patru.
b. **Punct slab:** Algoritmul se blochează în primul minim local pe care îl găsește.
c. **Concluzie:** Nu este recomandat pentru această problemă fără modificări (precum reporniri multiple), deoarece spațiul de căutare este complex.